

第 6 讲: 神经元与多层感知机

从大脑神经元到神经网络的基本结构

@ Frano

● FranSpace

本讲摘要 / Why it matters

神经网络这个名字并不是随便起的，它的灵感最早来自生物学中的神经元。人工神经元虽然远比真实大脑神经元简单，但它保留了一个核心思想：输入信息经过加权整合，再经过某种非线性处理后输出。许多人工神经元按层连接起来，就构成了多层感知机 (MLP)，它是最基本、最常见的一类神经网络，也是 PINN 中最常用的网络结构之一。

6.1 从大脑神经元说起

如果第一次接触神经网络，直接进入公式

$$z = \mathbf{w}^T \mathbf{x} + b$$

往往会显得有些突兀。更自然的方式，是先从“神经元”这个名字本身说起。

人脑中有大量神经元，它们通过复杂的连接网络进行信息传递。一个生物神经元通常包括：树突、细胞体和轴突。树突负责接收来自其他神经元的信号，细胞体对这些信号进行整合，若整合结果足够强，就会通过轴突向外发送信号。这个过程可以非常粗略地理解为：

1. 接收输入信号；
2. 对输入信号进行整合；
3. 判断是否激活；
4. 向外输出信号。

人工神经网络中的“神经元”正是借鉴了这个思路。它并不是对真实神经元的精确模

拟，而是一种高度抽象后的数学模型：输入若干数值，先做加权求和，再经过一个非线性函数，最后输出一个结果。

分析 / Insight

从学习角度看，生物神经元和人工神经元之间最重要的对应关系不是“形状像不像”，而是“信息处理过程像不像”。二者都可以概括为：接收输入 → 整合信息 → 产生输出。

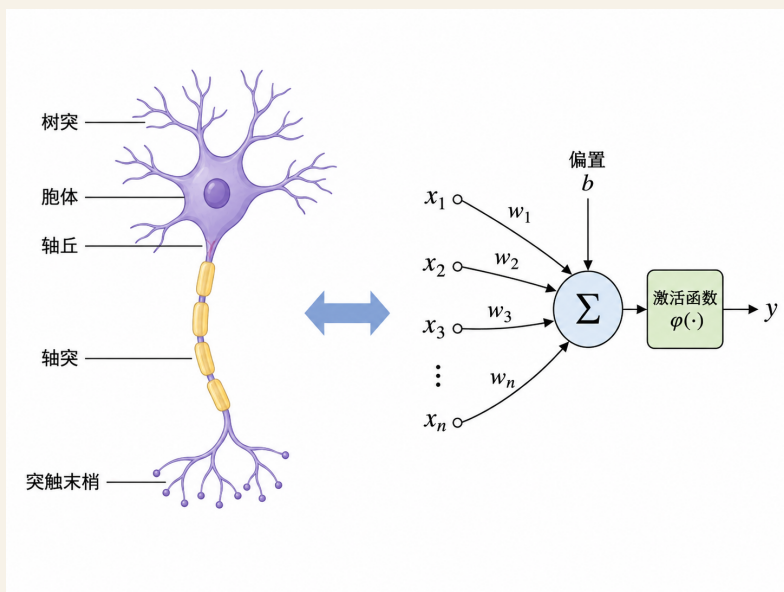


图 1: 生物神经元与人工神经元的对比示意图

6.2 什么是人工神经元

人工神经元是神经网络中最基本的计算单元。它接收一个输入向量

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T,$$

并为每个输入分配一个权重

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T.$$

然后对这些输入做加权求和，并加上偏置 b ：

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b. \quad (6.2.1)$$

这里的 z 称为线性组合结果，或者叫做神经元的“净输入”。

接下来，再通过一个激活函数 $\sigma(\cdot)$ 得到输出：

$$a = \sigma(z). \quad (6.2.2)$$

于是，一个完整的人工神经元就可以写成：

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b). \quad (6.2.3)$$

这三个部分各自的作用如下：

- 输入 $\mathbf{x}$ ：表示外部给定的信息，例如图像像素、房价特征、空间坐标等；
- 权重 $\mathbf{w}$ ：表示不同输入的重要性；
- 偏置 b ：表示神经元的“平移量”或“阈值修正”；
- 激活函数 σ ：引入非线性，使网络能够表示更复杂的关系。

Example / 例子

假设我们要根据三个特征预测某个结果，这三个特征分别是 x_1, x_2, x_3 。一个人工神经元可以写成：

$$z = 0.5x_1 + 1.2x_2 - 0.8x_3 + 0.3, \quad a = \tanh(z).$$

这表示：神经元先把三个输入按不同权重组合起来，再通过 $\tanh$ 变成最终输出。若训练后发现某个权重很大，说明对应输入对该神经元更重要。

6.3 为什么需要权重和偏置

很多初学者会问：为什么非要写成

$$\mathbf{w}^T \mathbf{x} + b$$

这种形式？为什么不能直接把输入丢进一个函数里？

一个很重要的原因是：这种形式既简单，又具有足够的表达基础。权重 $\mathbf{w}$ 负责控制不同输入的重要程度，偏置 b 负责调整输出的位置。

从几何上看，若没有激活函数，仅考虑

$$z = \mathbf{w}^T \mathbf{x} + b,$$

它表示的是一个线性变换。以二维情况为例，

$$z = w_1x_1 + w_2x_2 + b$$

对应一个平面中的直线。加入激活函数后，神经元就可以在这个线性基础上进一步产生非线性表达。

备注 / Note

权重和偏置都是模型参数。训练神经网络的核心任务之一，就是通过反向传播和优化器不断调整这些参数，使模型输出更接近目标值。

6.4 一个神经元为什么不够

单个神经元的能力是有限的。即使加入激活函数，它本质上也只是对输入做一次“加权求和 + 非线性变换”。如果问题很复杂，仅靠一个神经元通常远远不够。

例如：

- 识别图片中的猫和狗，不可能只依赖一个神经元；
- 预测复杂的非线性函数，也不可能只依赖一个神经元；
- 在 PINN 中近似位移场、温度场、相场变量等连续函数，更需要多个神经元协同工作。

因此，我们需要把许多神经元组织起来，形成层，再把多层连接起来，形成更强大的网络结构。

6.5 从神经元到层

当多个神经元同时接收同一组输入时，就可以看作形成了一层。假设某一层有 m 个神经元，那么它们会产生一个 m 维输出向量：

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix}.$$

若把所有神经元的权重放在一起，就可以写成矩阵形式：

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad (6.5.1)$$

然后通过激活函数得到：

$$\mathbf{a} = \sigma(\mathbf{z}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}). \quad (6.5.2)$$

这里：

- $\mathbf{W}$ 是权重矩阵；

- $\mathbf{b}$ 是偏置向量;
- σ 对向量中的每个分量分别作用。

用矩阵形式写的好处是：结构更清晰，也更适合计算机实现。

6.6 多层感知机 MLP 的基本结构

多层感知机 (Multi-Layer Perceptron, MLP) 是最经典的前馈神经网络之一。所谓“多层”，是指它至少包含一层隐藏层；所谓“前馈”，是指信息从输入层流向输出层，中间不形成回路。

一个典型的 MLP 包括三部分：

- 输入层：负责接收原始输入；
- 隐藏层：负责提取特征和构造非线性表达；
- 输出层：负责输出最终预测结果。

设第 l 层的输入为 $\mathbf{a}^{(l-1)}$ ，则第 l 层的输出通常写成：

$$\mathbf{a}^{(l)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right). \quad (6.6.1)$$

如果网络有 L 层，那么从输入到输出的过程就是不断重复上述变换。

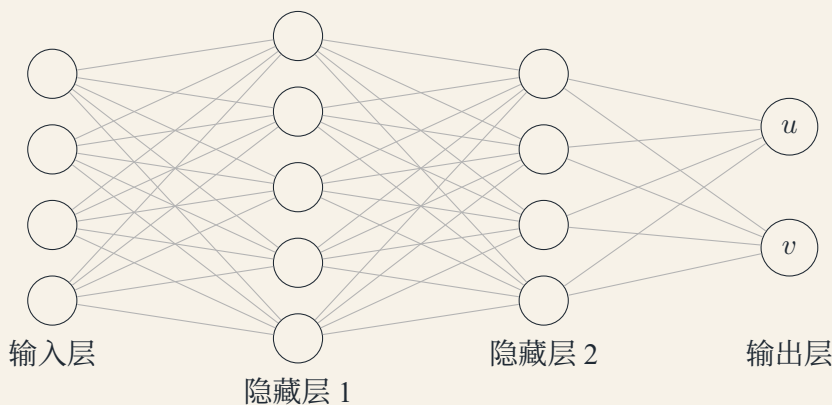


图 2: 一个典型的多层感知机 (MLP) 结构示意图

6.7 输入层、隐藏层和输出层分别在做什么

这三个名字虽然经常出现，但初学时很容易只记住名字，没有真正理解含义。

6.7.1 输入层

输入层本身通常不做复杂运算，它主要负责把外部信息送入网络。例如：

- 对图像问题，输入可能是像素值；
- 对表格数据问题，输入可能是房价、年龄、收入等特征；
- 对 PINN，输入通常是空间坐标 (X, Y) ，或时空坐标 (X, Y, t) 。

● 6.7.2 隐藏层

隐藏层是网络真正“加工信息”的地方。每个隐藏层都会对输入做线性变换和非线性变换，逐步提取和重组特征。隐藏层越多，理论上越有可能表达更复杂的函数。

● 6.7.3 输出层

输出层负责生成最终预测结果。例如：

- 分类问题输出类别概率；
- 回归问题输出连续数值；
- PINN 中输出位移、温度、相场变量等物理场。

Example / 例子

对于二维固体力学 PINN，一个很常见的设计是：

- 输入层： (X, Y) ；
- 隐藏层：若干层全连接层；
- 输出层： (u, v) 。

这样网络输出的就是每个坐标点上的位移预测。后续再通过自动微分计算应变、应力和 PDE 残差。

6.8 为什么叫“多层感知机”

“感知机”最初是较早的一种人工神经元模型。“多层感知机”表示：不再只有一层简单的感知机，而是通过多层叠加来提高表达能力。

从函数逼近角度看，MLP 的一个重要特点是：它可以逼近相当广泛的一类函数。这也是为什么它虽然结构简单，却一直是神经网络中的基础结构之一。

分析 / Insight

你可以把 MLP 理解成一个“通用函数逼近器”。它并不一定最适合所有任务，但它结构清晰、实现简单、适用范围广，因此在入门学习和 PINN 中都非常常见。

6.9 深度和宽度是什么意思

讨论神经网络结构时，经常会听到两个词：深度和宽度。

- 深度：指隐藏层的层数；
- 宽度：指每层神经元的数量。

例如：

- 一个“3 层、每层 20 个神经元”的网络，通常表示有 3 个隐藏层，每层宽度为 20；
- 若网络更深，说明层数更多；
- 若网络更宽，说明每层神经元更多。

增加深度和宽度通常可以增强模型表达能力，但代价是：

- 参数数量增加；
- 训练时间变长；
- 显存占用增大；
- 过拟合风险增大；
- 自动微分成本上升。

备注 / Note

在 PINN 中，并不是网络越大越好。因为 PINN 还要额外计算导数、残差和边界条件，网络过深或过宽可能会显著增加训练难度和计算成本。

6.10 MLP 在 PINN 中为什么常用

你后面学习 PINN 时会反复看到 MLP，这是因为它有几个非常适合 PINN 的特点：

- 结构简单，容易实现；
- 输入和输出维度灵活；

- 适合近似连续函数；
- 可以直接配合自动微分计算一阶、二阶导数；
- 便于和 PDE 残差、边界条件 loss 结合。

例如，在固体力学 PINN 中，常常用 MLP 来近似位移场：

$$\mathbf{u}_\theta(\mathbf{X}) = \begin{bmatrix} u_\theta(\mathbf{X}) \\ v_\theta(\mathbf{X}) \end{bmatrix}. \quad (6.10.1)$$

再由自动微分得到位移梯度：

$$\frac{\partial \mathbf{u}_\theta}{\partial \mathbf{X}}, \quad (6.10.2)$$

从而构造变形梯度

$$\mathbf{F} = \mathbf{I} + \frac{\partial \mathbf{u}_\theta}{\partial \mathbf{X}}. \quad (6.10.3)$$

接着再根据本构关系计算应力和 PDE 残差。因此，MLP 在 PINN 中并不仅仅是“输出一个数”，而是承担了“近似整个物理场函数”的任务。

6.11 一个完整的小例子

为了把神经元和 MLP 的概念串起来，我们看一个简单例子。假设要建立一个二维 PINN，用来近似二维位移场。

输入取为：

$$\mathbf{x} = \begin{bmatrix} X \\ Y \end{bmatrix}.$$

第一隐藏层可写成：

$$\mathbf{a}^{(1)} = \sigma \left(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right).$$

第二隐藏层可写成：

$$\mathbf{a}^{(2)} = \sigma \left(\mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)} \right).$$

输出层可写成：

$$\hat{\mathbf{u}} = \mathbf{W}^{(3)} \mathbf{a}^{(2)} + \mathbf{b}^{(3)} = \begin{bmatrix} u_\theta(X, Y) \\ v_\theta(X, Y) \end{bmatrix}.$$

这说明：从两个输入坐标出发，经过多层神经元的非线性组合，最终输出两个位移分量。这就是一个最基本的 MLP 型 PINN 网络。

6.12 常见误区

初学神经网络时，关于神经元和 MLP 常有几个误区：

- 误区一：神经网络真的在模拟大脑。

其实神经网络只是受生物神经元启发的数学模型，并不是对真实大脑的精确复刻。

- 误区二：一个神经元就很强大。

单个神经元能力很有限，真正强大的是大量神经元按层组合形成的网络结构。

- 误区三：隐藏层越多越好。

网络太深会增加训练难度和计算成本，在 PINN 中尤其明显。

- 误区四：MLP 只能做分类。

MLP 可以做分类、回归，也可以在 PINN 中近似连续物理场函数。

6.13 一页总结

- 人工神经元的灵感来源于生物神经元的信息处理方式：接收输入、整合信息并输出结果。

- 一个人工神经元通常写成：

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b).$$

- 其中 $\mathbf{x}$ 是输入， $\mathbf{w}$ 是权重， b 是偏置， σ 是激活函数。
- 多个神经元组成一层，多层叠加就形成多层感知机 MLP。
- 第 l 层的输出通常写成：

$$\mathbf{a}^{(l)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right).$$

- MLP 包括输入层、隐藏层和输出层，是最基本、最常见的神经网络结构之一。
- 深度表示层数，宽度表示每层神经元数量。增加深度和宽度可以增强表达能力，但也会增加训练成本。
- 在 PINN 中，MLP 常用于近似位移场、温度场或相场变量等连续函数，并配合自动微分计算导数和 PDE 残差。